Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Rozšíriteľná knižnica pre správu Docker kontajnerov implementovaná v Go

2026

Samuel Táborský

Vedoucí práce:
Mgr. Tomáš Urbanec, Ph.D.

Studijní program: Informatika,
Specializace: Programování a vývoj software

Bibliografické údaje

Autor:	Samuel Táborský
Název práce:	Rozšíriteľná knižnica pre správu Docker kontajnerov implementovaná v Go
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Informatika, Specializace: Programování a vývoj software
Vedoucí práce:	Mgr. Tomáš Urbanec, Ph.D.
Počet stran:	14
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	slovenský

Bibliographic info

Author:	Samuel Táborský
Title:	An extensible library for Docker Container management implemented in Go
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Tomáš Urbanec, Ph.D.
Page count:	14
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Slovak

Anotácia

Táto bakalárska práca sa zaoberá návrhom a implementáciou knižnice v jazyku Go na správu Docker kontajnerov. Opisuje teoretické východiská kontajnerizácie a prístup cez Docker API, pričom sa sústreďuje na modulárnu architektúru umožňujúcu rozšíriteľnosť aj podporu rôznych kontajnerových runtime. Praktická časť práce zahŕňa implementáciu knižnice a sprievodnej ukážkovej aplikácie pre správu kontajnerov, sledovanie udalostí a monitoring prostriedkov. Výsledné riešenie je overené testami a predstavuje základ pre ďalší vývoj a rozšírenie v open-source prostredí.

Synopsis

This bachelor's thesis focuses on the design and implementation of a Go-based library for Docker container management. It outlines the fundamentals of containerization and Docker API access, with emphasis on a modular architecture enabling extensibility and support for multiple container runtimes. The practical part includes the implementation of the library and an accompanying demo application for container management, event monitoring, and resource tracking. The solution is validated by testing and provides a foundation for further development and open-source extension.

Kľúčové slová: Docker; kontajnerizácia, knižnica; jazyk Go

Keywords: Docker; containerization; library; Go language

Poďakovania

Odevzdaním tohto textu jeho autor/ka cestne vyhlasuje, že celú prácu vrátane príloh vypracoval/a samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1 Úvod **6**
1.1 Motivácia a ciele práce 6
1.2 Vymedzenie problému 7
1.3 Prehľad existujúcich riešení 7
1.4 Prínos a očakávania 8

2 Teoretické základy **9**
2.1 Virtualizácia a kontajnerizácia 9
2.2 Docker a jeho architektúra 9
2.3 Programovací jazyk Go a jeho ekosystém 9
2.4 Softvérové architektúry a rozšíriteľnosť knižníc 9

3 Analýza problému **9**
3.1 Požiadavky na systém 9
3.2 Prehľad dostupných knižníc a nástrojov 9
3.3 Návrh architektúry riešenia 10

4 Návrh a implementácia riešenia **10**
4.1 Návrh knižnice 10
4.2 Implementácia Docker klienta 10
4.3 Testovanie a validácia 10
4.4 Implementácia ukážkovej aplikácie 10

5 Možné budúce rozšírenia **10**
5.1 Podpora iných runtimes (Podman, containerd! 10
5.2 Správa zväzkov a sieťovanie 10
5.3 Open-source pokračovanie 10

Záver **11**

Conclusions **12**

6 Obsah elektronických dát **12**

Zoznam skratiek **14**

1 Úvod

V posledných rokoch sa kontajnerizácia stala základným stavebným kameňom vývoja a nasadzovania moderného softvéru. Na tomto koncepte je postavené veľké množstvo moderných produktov a nástrojov, od cloudových platforiem použiteľných pre nasadenie softvéru do produkčného prostredia, nástrojov na jednoduché zdieľanie kompletných vývojových prostredí medzi softvérovými inžiniermi a programátormi, až po škálovateľné systémy založené na architektúre mikroslužieb (microservice architecture). Väčšina súčasných webových aplikácií a webstránok už je nasadených v distribuovanom systéme postavenom z kontajnerov, čo prináša možnosť veľmi jednoduchého škálovania, vyrovnávania záťaže (load balancing), zálohovania a zaručenia vysokej dostupnosti. Prechod z tradičnej hardvérovej virtualizácie na kontajnerizáciu uľahčil a zefektívnil každú časť softvérového vývojového procesu, počínajúc reprodukovateľnými vývojovými prostrediami, cez integráciu CI/CD postupov do vývoja, až po nasadenie a správu v produkcii.

Jedna z najznámejších technológií spojených s kontajnerizáciou je **Docker**. Poskytuje komplexné rozhranie na správu životného cyklu kontajnerov a obrazov, ako aj na konfiguráciu sietí, pripájanie lokálnych zväzkov (volumes) a monitorovanie udalostí. Docker však nie je jediná technológia v tomto ekosystéme. Okrem neho existujú ďalšie nástroje s rôznymi architektúrami. Medzi ďalšie známe produkty a technológie patrí napríklad "daemonless"**Podman**, distribuované orchestračné platformy ako **Kubernetes**, alebo nízkoúrovňový runtime engine **containerd**. Táto rôznorodosť prístupov a abstrakčných vrstiev so sebou prináša komplexné prostredie, s ktorým sa súčasní vývojári musia vysporiadať.

1.1 Motivácia a ciele práce

Pre lokálne vývojové prostredia vývojári často preferujú vysokoúrovňové riešenia (ako napríklad Docker), postavené na princípe jednoduchého používania a poskytujúce napríklad konzolové (CLI) alebo grafické rozhranie. V produkčnom prostredí a orchestrácii sú však uprednostňované nástroje oveľa nižšej úrovne, ako napríklad ľahký **containerd**.

Tento fakt prináša rôzne problémy: diametrálne odlišné rozhranie (API) a jeho zložitosť, spôsob práce, a nutnosť ovládať niekoľko nástrojov. Kým knižnica (SDK) pre Docker je postavená na jednoduchších synchrónnych akciách, containerd je omnoho modulárnejšie a vyžaduje od vývojára manuálnu správu niekoľkých komponentov (napr. snapshoty, namespaces, tasky a iné).

Zároveň vyvstáva problém tzv. "Vendor lock-in", čo v praxi znamená neprenositeľnosť aplikácie, postavenej napríklad na Docker SDK, na systémy, kde beží iba containerd alebo Kubernetes. Ak je žiadané podporovať oba druhy systémov, je nutné udržovať viaceré odlišné kusy kódu, čo prináša ďalšiu komplexnosť.

Mojou motiváciou bolo vytvoriť jednoduché a elegantné unifikované rozhranie v jazyku Go, ktoré by vývojárom umožnilo použiť jeden kód na prácu s kontaj-

nermi bez ohľadu na to, aký nástroj sa bude skrývať v pozadí. Cieľom tejto práce je analyzovať a porovnať existujúce riešenia na správu kontajnerov a ich architektúru. Na základe tejto analýzy navrhnúť jednotné rozhranie a abstrakčnú vrstvu, ktorá skryje implementačné rozdiely jednotlivých enginov pred koncovým používateľom. Nasleduje naprogramovanie samotnej knižnice v jazyku Go, ktorá bude obsahovať modul pre prácu s nástrojom Docker, a zároveň jasný a jednoduchý spôsob, ako knižnicu rozšíriť o ďalšie kontajnerové runtime. Nakoniec vytvorenú knižnicu otestovať a pripraviť ukážkový program demonštrujúci jej použitie.

1.2 Koncept riešenia

Základom knižnice bude jednotné rozhranie obsahujúce funkcie na správu životného cyklu kontajnerov a obrazov, konfiguráciu sietí, pripájanie lokálnych zväzkov a monitorovanie a reakcie na rôzne udalosti. Z nástrojov **Docker**, **Podman** a **containerd** bude vybraná čo najširšia spoločná funkcionalita, ktorá bude tvoriť toto rozhranie. Zároveň bude knižnica obsahovať jednoduché dátové typy a štruktúry, ktoré odtienia konkrétne implementačné detaily, vstupné a návratové typy a spôsoby predávania argumentov funkciám jednotlivých technológií.

Vďaka tomu bude mať vývojár dostupný nástroj, pomocou ktorého bude možné implementovať aplikácie bez potreby písania kódu pre rôzne runtime, pracovania s rozdielnymi dátovými typmi a spôsobmi riadenia toku programu. Takto implementovaná knižnica bude predstavovať základ pre jednoduchý, jednotný a rozšíriteľný systém, ktorý umožní nielen vývoj platformovo nezávislých aplikácií, ale napríklad aj ich testovanie. Vývojár bude mať možnosť vytvárať a spravovať kontajnery so službami, ktoré sú potrebné počas behu testu, ako sú mimo iného databázové systémy alebo autentifikačné a logovacie nástroje.

1.3 Prehľad existujúcich riešení

Momentálne najčastejší prístup, akým vývojári integrujú do svojho projektu kontajnerové runtimes, je pomocou využitia ich oficiálnych klientských knižníc (SDK). Pre Docker existuje **Moby SDK**, pre Podman je dostupné **Podman SDK** a pre containerd existuje oficiálna knižnica slúžiaca pre tvorcov iných nástrojov (napr. samotný Docker používa túto knižnicu). Tieto riešenia však neposkytujú žiadnu spoločnú abstrakciu, takže je nutné udržovať samostatný kód a vetvy logiky pre každé z nich.

Kubernetes definoval pomocou gRPC protokolu spoločné rozhranie **Kubernetes CRI (Container Runtime Interface)**. Toto rozhranie funguje na princípe klient-server. Na každom serveri (node) toto rozhranie používa agent **Kubelet**. Ten pomocou neho komunikuje s nízkoúrovňovým engine (napr. **containerd**, **cri-o**, alebo v minulosti **dockershim**), ktorý toto rozhranie implementuje. CRI je síce extrémne robustné riešenie, ale je určené pre masívnu orchestráciu. To pre vývojára znamená implementáciu gRPC klienta a komunikáciu cez CRI, čo je pri písaní menších projektov, skriptovaní, automatizácií či vývoju CLI nástrojov

príliš komplikované riešenie.

Ďalší populárny projekt v ekosystéme jazyka Go je **Testcontainers**. Táto knižnica umožňuje vývojárom priamo v jednotkových testoch (unit tests) programaticky naštartovať napr. databázu, spustiť testy a nakoniec kontajner zahodiť. Je však viazaná na kontext softvérového testovania. Obsahuje logiku pre čakanie na rôzne operácie (tzv. **waiting strategies**), automatické mazanie kontajnerov pomocou privilegovaného **Ryuk** kontajneru a iné nástroje mierené na testovanie softvéru. Nie je to všeobecná abstrakčná knižnica určená na produkčnú správu životného cyklu kontajnerov.

Podman poskytuje službu, ktorá emuluje Docker REST API (často na rovnakom sockete), čo umožňuje staršie nástroje napísané pre Docker používať s Podman. Toto však rieši problém fragmentácie rozhraní a dátových typov iba pre tieto dve kontajnerové runtime, a to iba na úrovni operačného systému, nie v programovom kóde. Stále teda chýba nástroj, ktorým je možné jedným kódom ovládať napríklad Docker, Podman a containerd zároveň.

Z prehľadu existujúcich riešení vyplýva, že hoci existujú snahy o štandardizáciu (ako napríklad gRPC rozhranie CRI v Kubernetes), tieto riešenia sú pre mnohé menšie a stredne veľké softvérové projekty príliš robustné a ťažkopádne. Na druhej strane, priame použitie natívnych SDK knižníc vedie k duplicite kódu a silnej závislosti na konkrétnom poskytovateľovi. V súčasnosti v ekosystéme jazyka Go absentuje ľahká, všeobecne použiteľná knižnica, ktorá by poskytla vývojárom jednotné rozhranie a abstrahovala by radikálne rozdiely medzi rozhraniami rôznych kontajnerových runtime bez nutnosti implementácie robustných architektonických vzorov z prostredia Kubernetes.

1.4 Prínos a očakávania

Hlavným prínosom tejto práce je poskytnutie nástroja, ktorý odstráni závislosť aplikácií od jedného konkrétneho kontajnerového runtime. Vývojári, ktorí túto knižnicu použijú, nebudú pevne zviazaný napríklad iba s Docker, ale budú mať v rukách spôsob, ako rovnaký kód použiť nezávisle na systéme, na ktorom bude ich aplikácia nasadená, prípadne nezávisle na rozhodnutí zmeniť runtime, ktorý beží v pozadí. Práca prinesie čisté, unifikované API v jazyku Go. Vďaka abstrakcii konkrétnych implementačných detailov jednotlivých runtime sa rapídne zníži čas potrebný na vývoj nástrojov pracujúcich s kontajnermi. Zároveň práca ponúka "lightweight"riešenie pre vývojárov, pre ktorých je implementácia komplexného CRI (Container Runtime Interface) cez gRPC protokol zbytočne náročná a neprimeraná ich potrebám.

Projekt bude obsahovať centrálny balíček obsahujúci unifikované rozhranie pre operácie nad kontajnermi a obrazmi, a zároveň dátový model štruktúr, ktoré toto rozhranie využíva. Taktiež bude obsahovať aspoň jeden balíček pre kontajnerový runtime Docker, použiteľný na komunikáciu s Docker API.

2 Teoretické základy

2.1 Virtualizácia a kontajnerizácia

V tejto kapitole popíšem históriu kontajnerizácie a jej vzniku. Zameriam sa takisto na porovnanie s bežnou virtualizáciou, prednesiem výhody a nevýhody oboch prístupov, ako aj moderné použitie kontajnerov v cloudových službách.

2.2 Docker a jeho architektúra

V tejto kapitole predstavím Docker a Docker Engine, jeho architektúru, históriu a komponenty. Popíšem ako vyzerá REST API, s ktorým bude moja knižnica komunikovať, a prednesiem technické možnosti, ktoré Docker poskytuje na správu kontajnerov.

2.3 Programovací jazyk Go a jeho ekosystém

Predstavím jazyk Go ako moderné pokračovanie nízkoúrovňových jazykov typu C, jeho vlastnosti a špecifické možnosti. Popíšem štruktúru programov napísaných v jazyku Go, používanie modulov a správne praktiky pre písanie efektívneho, znovupoužiteľného a rozšíriteľného kódu.

2.4 Softvérové architektúry a rozšíriteľnosť knižníc

V tejto kapitole popíšem rôzne architektúry softvéru, ich výhody a nevýhody. Rovnako sa zameriam aj na praktiky, ktoré umožňujú zjednodušiť vývoj a následné udržovanie a rozširovanie softvérových aplikácií. Skrátka predstavím aj spôsob, ktorým docielim rozšíriteľnosť mojej knižnice o podporu ďalších backendov, ako je napríklad Podman.

3 Analýza problému

3.1 Požiadavky na systém

V tejto podkapitole vymedzím požiadavky na funkčnosť aplikácie (ako napríklad možnosti správy kontajnerov, monitorovanie ich stavu atď.), ako aj požiadavky na štruktúru aplikácie, jej rozšíriteľnosť a čistotu kódu.

3.2 Prehľad dostupných knižníc a nástrojov

Predstavím všetky nástroje a knižnice, ktoré bude moja aplikácia využívať, na čo ich bude využívať a prečo niektoré časti nebudem písať ja. Ukážem rozdiely medzi REST API a SDK, a takisto v krátkosti predstavím nástroje a knižnice, ktoré by mohli byť použité pre rozšírenie na iné kontajnerové runtimes.

3.3 Návrh architektúry riešenia

Popíšem architektúru celej aplikácie, jej rozdelenie na oddelené vrstvy a dôvody na tieto rozhodnutia. Predstavím základné rozhranie aplikácie, ktoré bude spoločné pre všetky runtimes. Rovnako popíšem dizajn, ktorý mi umožňuje v budúcnosti aplikáciu rozšíriť.

4 Návrh a implementácia riešenia

4.1 Návrh knižnice

Popíšem štruktúru knižnice a repozitára, rozdelenie na samostatné časti, ďalej predstavím typy a verejné rozhranie (API). Vysvetlím moje použitie submodulov namiesto reexportu v hlavnom module, ako aj možnosť prejsť v budúcnosti na tento spôsob.

4.2 Implementácia Docker klienta

Opíšem implementáciu klienta, ktorý komunikuje s Docker Engine cez REST API a SDK. Ukážem funkcionalitu, ktorú klient umožňuje (správa životného cyklu, monitorovanie, sledovanie udalostí - eventov, správa kontajnerov).

4.3 Testovanie a validácia

Ukážem, akým spôsobom je aplikácia testovaná a ktoré jej časti sú pokryté. Použijem unit testy a integračné testy.

4.4 Implementácia ukážkovej aplikácie

Predstavím jednoduchú ukážkovú aplikáciu, ktorá demonštruje spôsob a možnosti použitia tejto knižnice.

5 Možné budúce rozšírenia

5.1 Podpora iných runtimes (Podman, containerd!

5.2 Správa zväzkov a sieťovanie

5.3 Open-source pokračovanie

Záver

Závěr práce v „českém“ jazyce.

Conclusions

Thesis conclusions in “English”.

6 Obsah elektronických dát

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

`text/`
: Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

`README.*`
: Textový soubor (s příponou např. `.txt`) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

`*`
: Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,
- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwarového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovolují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru `README.*`.